Syntax Errors

A parser has two jobs

1. Given a *valid* sequence of tokens, *produce a corresponding syntax tree*

That's what our code does

2. Given an *invalid* sequence of tokens, report an error

Modern IDEs **depend** on that parser reporting errors, and the parser is *constantly reparsing code* as you type, so that it can report errors in *real time*

This is done for *syntax highlighting*, *auto-completion*, *error lines*, and more

Hard requirements

A parser **must**:

1. *Detect and report the error*

If it can't find the error, and makes a *malformed syntax tree* which is then sent to the *interpreter*

Undefined behavior can happen which can range from *crashes* to *silent bugs* to *security vulnerabilities*

2. *Avoid crashing or hanging*

Syntax errors are **common and expected**, and languages should be robust enough to handle them gracefully, without crashing or hanging

Because while the source code you write may not be *valid* code, it's still *valid input* because it's something the parser should be able to handle

Soft Requirements

Other requirements include

1. *Be fast*, computers are thousands of times faster than they were when parsers *first started being developed*. Speed is no longer as much of an issue

> But programmer expectations have also increased

Programmers expect to be able to type a key, and have the parser re-parse the code and report any errors in *real time*

2. *Report as many distinct errors as there are*, aborting after the first error is easy to implement, but it's annoying for users

3. *Minimize cascaded errors*, once a single error is found, the parser no longer has a clear picture of the code

This means it might report a *bunch of errors* that are all caused by the same underlying issue, which can be overwhelming for users

Panic mode Error Recovery

Of all the recovery techniques devised, the simplest and most consistent is **panic mode** error recovery

As soon as the parser detects an error, it enters *panic mode*

It knows that *at least one token doesn't make sense*

And before it can go back to parsing, it needs to *get its current state* and the *sequence of forthcoming tokens*

So that the next token does match the rule being parsed

This is called **synchronization**

Synchronization

To do that, we select a rule in the grammar that will mark the **synchronization point**

The parser fixes its parsing state by *jumping out of any nested productions until it gets back to that rule*

Then it synchronizes the token stream by *discarding tokens* until it reaches one that can appear at that point in the rule

Traditionally, we synchronize *between statements* (between lines), but we don't have statements right now

So we won't be doing any synchronization until we have statements

In summary

1. Parser encounters an error, and enters panic mode
2. It jumps out of any nested productions until it gets back to the synchronization point
3. It discards tokens until it finds one that can appear at the synchronization point
4. It exits panic mode and resumes normal parsing

Run Through

Run Through

Let’s run through a statement and run through how our *current* program handles it

```
1   !true == (5 + 3 ⩽ 10 - 2)
```

this style of question will be in the exam

```
-5 + 10 ⩾ -(-5)
```